

OceanBase × LangChain Community

Course Series

Building Agentic RAG with LangChain & OceanBase

Episode 2

RAG Fundamentals & Architectures

Haili Zhang · LangChain Ambassador





LangChain Community



Haili Zhang

LangChain Ambassador



GitHub

@webup



Twitter

@zhanghaili0610



Bilibili

沧海九粟

What We'll Cover

From fundamentals to production-ready RAG systems



RAG Fundamentals

Retrieval

Why LLMs need retrieval

Theory



Knowledge Base

Vector DB

Building searchable knowledge

Practice



RAG Patterns

2-Step & Agentic

Different architectures

Comparison

RAG Fundamentals



Beyond Context Windows

Breaking through LLM limitations with retrieval

What is RAG?

Retrieval-Augmented Generation combines external knowledge retrieval with LLM generation

⚠ LLM Limitations

- ✗ **Finite Context Window**
Cannot process entire corpora at once
- ✗ **Static Knowledge**
Training data frozen at a point in time
- ✗ **No Source Attribution**
Cannot cite or verify where information comes from

✓ RAG Solution

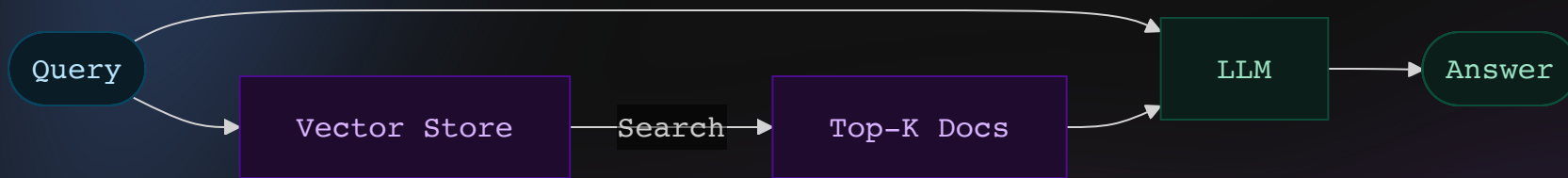
- ✓ **External Knowledge Retrieval**
Fetch relevant information at query time
- ✓ **Grounded Answers**
Generate responses based on retrieved context
- ✓ **Traceable Sources**
Citations and provenance for every answer



RAG = LLM Intelligence + External Knowledge + Source Attribution

RAG Pipeline

The typical flow from query to answer



 RAG bridges the gap between LLM knowledge and external data

Final Prompt Structure

System:

Answer based on the context provided. If uncertain, say so.

Context:

[Doc 1] Nike's 2023 revenues ... [Doc 2] Financial highlights ... [Doc 3] Revenue breakdown ...

Query:

RAG Architectures

Different approaches to implementing RAG

2-Step RAG

Query → Retrieve → Generate

- ✓ High control
- ✓ Predictable latency
- ⚠ Always retrieves

Agentic RAG

Query → Agent Decision
↓
Tool Usage (0-N times)
↓
Answer

- ✓ Flexible retrieval
- ✓ Multi-step reasoning
- ⚠ Variable latency

Corrective RAG

Query → Enhanced Query
↓
Retrieval + Validation
↓
Generate + Verify

- ✓ Self-correction
- ✓ Answer validation
- ⚠ More complex



Choose your pattern: 2-Step for predictability, Agentic for flexibility, Corrective for quality

Knowledge Base

Building Knowledge

Transform documents into searchable intelligence

Knowledge Base Pipeline

From raw documents to searchable vectors



Document Loaders

Ingest data from sources



Text Splitters

Break into chunks



Embeddings

Convert to vectors



Vector Store

Store and search

Sources → Loaders → Documents → Splitters → Chunks → Embeddings → Vectors → Vector Store

Example: Nike 10-K Report

Real-world knowledge base built from financial document

Data Source

Nike's 2023 Annual Report (Form 10-K)

Pages	Size	Characters
-------	------	------------

107	2.3 MB	376K
-----	--------	------

Embeddings

Model	BAAI/bge-m3
-------	-------------

Dimension	1024
-----------	------

Vector Storage	~1.8 MB
----------------	---------

Text Processing

Chunk Size	1000
------------	------

Overlap	100
---------	-----

Total Chunks	460
--------------	-----

Similarity Search

Example Query

```
"What were Nike's total revenues in fiscal 2023?"
```

→ Returns relevant financial data chunks

Document Loaders

Ingest documents from various sources into standardized Document objects

PDF Loader

```
from langchain_community.document_loaders import PyPDFLoader

# Load PDF document
loader = PyPDFLoader("nike-10k.pdf")
documents = loader.load() # 107 pages

# Document structure
print(documents[0].page_content)
# "In fiscal 2023, NIKE, Inc..."

print(documents[0].metadata)
# {'source': 'nike-10k.pdf', 'page': 0}
```

- ✓ Page-by-page parsing
- ✓ Rich metadata
- ✓ Preserves structure

Other Loaders

```
# Web pages
from langchain_community.document_loaders import WebBaseLoader
loader = WebBaseLoader("https:// ... ")
docs = loader.load()

# Markdown files
from langchain_community.document_loaders import \
    UnstructuredMarkdownLoader
loader = UnstructuredMarkdownLoader("doc.md")

# Also: CSV, JSON, Notion, S3,
# Google Drive, Confluence, etc.
```

- ✓ 100+ data sources
- ✓ Unified interface
- ✓ Custom loaders

Text Splitters

Splitting Strategies

-  Text structure-based (paragraphs, sentences)
-  Length-based (tokens, characters)
-  Document structure-based (MD, HTML, JSON)



Recommended

Start with RecursiveCharacterTextSplitter – it balances context preservation and chunk size for most use cases



RecursiveCharacterTextSplitter

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Recommended for most use cases
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,          # Max characters per chunk
    chunk_overlap=100,        # Overlap preserves context
    separators=["\n\n", "\n", ". ", " ", ""] # Split hierarchy:
    # Try paragraph breaks (\n\n), then line breaks (\n), then sentences (.), then words ( )
)
chunks = text_splitter.split_documents(documents)
```

- ✓ Hierarchical splitting
- ✓ Maintains semantic flow
- ✓ Preserves context

Other Splitting Strategies

Specialized splitters for specific use cases

Length-Based Splitter

```
from langchain_text_splitters import CharacterTextSplitter

# Token-based splitting (model-aware)
text_splitter = CharacterTextSplitter.from_tiktoken(
    encoding_name="cl100k_base", # GPT
    chunk_size=100, # Max tokens
    chunk_overlap=0
)
texts = text_splitter.split_text(doc)
```

- ✓ Token-based splitting
- ✓ Consistent chunk sizes
- ✓ Model-aware limits

Document Structure-Based

```
# Markdown splitter
from langchain_text_splitters import MarkdownHeaderTextSplitter
splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=[
        ("#", "h1"), ("##", "h2")
    ]
)

# Also available: HTMLHeaderTextSplitter, CodeSplitter
```

- ✓ Preserves structure
- ✓ Semantic coherence
- ✓ Format-specific

Embeddings

Convert text into high-dimensional vectors that capture semantic meaning

```
register_embeddings_provider(  
    provider_name="siliconflow",  
    embeddings_model="openai-compatible",  
    base_url="https://api.siliconflow.cn/v1"  
)  
  
# Load embedding model with provider prefix  
embeddings = load_embeddings("siliconflow:BAAI/bge-m3")  
  
# Generate vector for a query  
vector = embeddings.embed_query("AI technology")  
# Returns: [0.043, -0.024, 0.127, ...]  
# 1024 dimensions  
  
# Batch embed multiple documents  
docs = ["Document 1", "Document 2", "Document 3"]  
vectors = embeddings.embed_documents(docs)  
# Returns: [[...], [...], [...]]  
# List of 1024-d vectors
```



Popular Models

BAAI/bge-m3	1024d
text-embedding-3-small	1536d
text-embedding-3-large	3072d



Key Properties

- ✓ Semantic similarity
- ✓ Cosine distance
- ✓ Fixed dimension



langchain-dev-utils



A lightweight utility library for streamlining LangChain development. Provides unified model management, reducing boilerplate code and improving consistency.

- ✓ Unified model interface
- ✓ Multiple providers support
- ✓ Less boilerplate code

Key Functions

- `register_embeddings_provider`
Register an embedding model provider
- `load_embeddings`
Load an embedding model with provider prefix

Vector Store

Store embeddings with efficient similarity search

```
from langchain_oceanbase.vectorstores import OceanbaseVecStore

# Create vector store with HNSW index
vector_store = OceanbaseVectorStore(
    embedding_function=embeddings,
    table_name="knowledge_base",
    connection_args=connection_args,
    vidx_metric_type="cosine",
    index_type="HNSW",
)

# Add documents to vector store
ids = vector_store.add_documents(chunks)

# Similarity search
results = vector_store.similarity_search(
    query="What were Nike's revenues?",
    k=3 # Return top 3 results
)
```

Vector Store Ecosystem

Key Features

- ✓ Distributed vector storage
- ✓ Hybrid search support
- ✓ Advanced metadata filtering

Index Types

HNSW	Fast ANN
IVF	Balanced
FLAT	Exact search

OceanBase × LangChain



Knowledge Base Pipeline: Complete Stack

The tools and libraries used to build a production-ready knowledge base

LangChain Ecosystem

Document loading and text processing

 PyPDFLoader PDF → Documents

 RecursiveCharacterTextSplitter

Chunk size 1000, overlap 100

Embeddings Management

Unified model interface with langchain-dev-utils

 langchain-dev-utils

Simplified provider registration

 Model: BAAI/bge-m3

Vector Store

Distributed vector storage with OceanBase

OceanBase Database

High availability & scalability

 langchain-oceanbase

Seamless LangChain integration

Complete Pipeline

End-to-end knowledge base construction

PDF → Loader → Documents

→ Splitter → Chunks

→ Embeddings → Vectors

→ OceanBase → ✓ Searchable KB

Building a Knowledge Base

Follow along with executable code cells demonstrating each step of the knowledge base construction

What's Covered

- ✔ Load Nike 10-K PDF with PyPDFLoader
- ✔ Split into chunks with RecursiveCharacterTextSplitter
- ✔ Generate embeddings using BGE-M3 via langchain-dev-utils
- ✔ Store vectors in OceanBase with HNSW indexing
- ✔ Perform similarity search and retrieval

Data Source

 Nike 10-K Annual Report (2023)
107 pages, 2.3 MB, financial statements and business operations

Learning Objectives

By the end of this notebook, you will:

- Load documents from various sources using document loaders
- Split text into optimized chunks using text splitters
- Generate embeddings with SiliconFlow's BGE-M3 model
- Store vectors in OceanBase with metadata
- Perform similarity search and retrieval

Knowledge Base Pipeline

A typical retrieval workflow:

```
plain
Sources → Document Loaders → Documents →
Text Splitters → Chunks → Embeddings →
Vector Store → Retriever
```

Each component is modular and can be swapped without rewriting the application logic.

Setup

RAG Patterns



Patterns that Scale

From prototype to production-ready RAG

2-Step RAG

Always retrieve before generating – predictable and controlled

```
# Create retriever
retriever = vector_store.as_retriever(k=3)

# Format retrieved docs
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

# Build RAG chain
rag_chain = (
    {
        "context": retriever | format_docs,
        "question": RunnablePassthrough()
    }
    | prompt
    | chat_model
)

# Use it
answer = rag_chain.invoke("What were Nike's total revenues in fiscal 2023?")
```

✓ Advantages

- ✓ Predictable execution flow
- ✓ Fast and deterministic
- ✓ Transparent system behavior

⚠ Disadvantages

- ✗ Always retrieves (even if unnecessary)
- ✗ Fixed number of documents (k)
- ✗ No dynamic reasoning about retrieval

💬 Q&A Demo

Question

"What were Nike's total revenues in fiscal 2023?"

Retrieved Context

[Page 30] In fiscal 2023, NIKE, Inc. achieved record Revenues of \$51.2 billion...

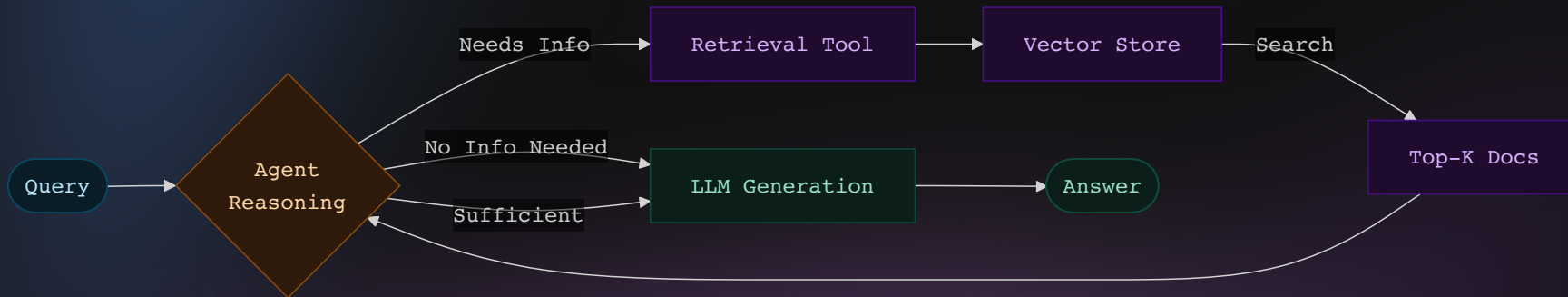
[Page 35] NIKE Brand revenues disaggregated by operating segment...

Generated Answer?

In fiscal 2023, Nike's total revenues were \$51.2 billion, representing a 10% increase...

Agentic RAG Pipeline

The agent decides if, when, and how many times to retrieve information based on the query.



The agent acts as a reasoning layer that decides whether to use retrieval tools. It can make multiple retrieval calls or skip retrieval entirely.



Agent Decision Process

Agentic RAG

Agent decides when and how to retrieve – flexible but variable

```
def search_knowledge_base(query: str) -> str:
    """Search Nike's 10-K report."""
    results = vector_store.similarity_search(query)
    return format_results(results)

# Create agent with tool
agent = create_agent(
    model=chat_model,
    tools=[search_knowledge_base],
    system_prompt=(
        "Use the search_knowledge_base tool when you need\n"
        "information about Nike's business"
    )
)

# Agent decides whether to search
result = agent.invoke({
    "messages": [{"role": "user", "content": "What is Nike's 2023 revenue?"}]
})
```

Agentic RAG Overview

✔ Advantages

- ✔ Flexible retrieval
- ✔ Multi-step reasoning
- ✔ Agent decides when to retrieve

❗ Disadvantages

- × Variable latency
- × Requires tool calling support
- × Complex debugging

💡 Scenario 1: Complex Question

Question

"Nike's 2023 revenues and income?"

✔ Used tool → Retrieved data

\$51.2B revenue, \$6.2B operating income

💡 Scenario 2: Simple Question

Question

"Hello, how are you?"

× No retrieval needed

Hello! I'm here to help you with any questions...

Comparing RAG Patterns

Characteristic / Scenario	 2-Step RAG	 Agentic RAG
 Control	 High	 Low
 Flexibility	 Low	 High
 Latency	 Predictable	 Variable
 Simple FAQ System	 Recommended	-
 Customer Support Chat	-	 Recommended
 Research & Analysis	-	 Recommended
 Code Q&A / Debugging	-	 Recommended

2-Step RAG Implementation

Build a complete RAG Q&A system with Nike's 10-K report – from retrieval to generation

What's Covered

- ✔ Build RAG chain with LangChain
- ✔ Compare responses with and without RAG
- ✔ Add source citations to answers
- ✔ Measure performance and latency
- ✔ Test with multiple Nike questions

patterns.two_step.key_features

- ⚡ Predictable latency (1 retrieval + 1 LLM call)
- 🎯 Grounded answers with Nike financial data
- 📊 Performance analysis and benchmarking

Learning Objectives

By the end of this notebook, you will:

- Understand the 2-Step RAG architecture
- Build a RAG chain with LangChain
- Create a Q&A system grounded in retrieved knowledge
- Compare responses with and without RAG
- Understand when to use 2-Step RAG vs other architectures

What is 2-Step RAG?

In **2-Step RAG**, retrieval always happens before generation:

```
plain
User Question → Retrieve Documents →
Generate Answer → Return to User
```

Characteristics:

Feature	Description
Control	✔ High - predictable execution flow
Flexibility	✗ Low - always retrieves before answering

Agentic RAG Implementation

Build an intelligent agent that decides when to retrieve from Nike's knowledge base

What's Covered

- ✔ Create retrieval tools for agents
- ✔ Build agent with LangChain v1 `create_agent()`
- ✔ Test intelligent decision-making
- ✔ Handle multi-step reasoning
- ✔ Compare Agentic vs 2-Step RAG

`patterns.agentic.key_features`

- 🤖 Agent decides when to retrieve (0 to N times)
- 🔄 Multi-step reasoning for complex questions
- 🎯 LangChain v1 agent with tool calling

Learning Objectives

By the end of this notebook, you will:

- Understand Agentic RAG architecture
- Build an agent that decides when to retrieve
- Create retrieval tools for agents
- Compare agentic vs 2-step RAG
- Handle multi-step reasoning with retrieval

What is Agentic RAG?

Agentic RAG combines retrieval with agent-based reasoning. Instead of always retrieving before answering, an LLM-powered agent reasons step-by-step and decides **when** and **how** to retrieve information.

```
plain
User Question → Agent (LLM) → Need info? →
Search Tool → Enough? → Generate Answer
```

↑

↓

|_____|

Key Insights

Key Takeaways

Essential principles for building production-ready RAG systems

What We Learned

Core concepts for building effective RAG applications

1

RAG solves LLM limitations

Combines external knowledge retrieval with generation for grounded, up-to-date answers

2

Knowledge base is the foundation

Document loaders → Text splitters → Embeddings → Vector store creates searchable knowledge

3

Different patterns for different needs

2-Step RAG for predictability, Agentic RAG for flexibility

4

Real-world example validates theory

Nike 10-K demonstrates complete pipeline from PDF to production Q&A system

Thank You ❤️

Build production-ready RAG systems with LangChain & OceanBase

